



Lecture 5

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

July 30, 2026

Code Optimization

- Précis writing

Code Optimization

- Précis writing
- Run faster

Code Optimization

- Précis writing
- Run faster
- Use fewer resources (memory, registers, space, fewer fetches)

Code Optimization

- Précis writing
- Run faster
- Use fewer resources (memory, registers, space, fewer fetches)
- Common optimization

Code Optimization

- Précis writing
- Run faster
- Use fewer resources (memory, registers, space, fewer fetches)
- Common optimization
 - ▶ Common sub-expression elimination

Code Optimization

- Précis writing
- Run faster
- Use fewer resources (memory, registers, space, fewer fetches)
- Common optimization
 - ▶ Common sub-expression elimination
 - ▶ Copy propagation

Code Optimization

- Précis writing
- Run faster
- Use fewer resources (memory, registers, space, fewer fetches)
- Common optimization
 - ▶ Common sub-expression elimination
 - ▶ Copy propagation
 - ▶ Dead code elimination

Code Optimization

- Précis writing
- Run faster
- Use fewer resources (memory, registers, space, fewer fetches)
- Common optimization
 - ▶ Common sub-expression elimination
 - ▶ Copy propagation
 - ▶ Dead code elimination
 - ▶ Code motion

Code Optimization

- Précis writing
- Run faster
- Use fewer resources (memory, registers, space, fewer fetches)
- Common optimization
 - ▶ Common sub-expression elimination
 - ▶ Copy propagation
 - ▶ Dead code elimination
 - ▶ Code motion
 - ▶ Strength reduction

Code Optimization

- Précis writing
- Run faster
- Use fewer resources (memory, registers, space, fewer fetches)
- Common optimization
 - ▶ Common sub-expression elimination
 - ▶ Copy propagation
 - ▶ Dead code elimination
 - ▶ Code motion
 - ▶ Strength reduction
 - ▶ Constant folding

Code Optimization

- Précis writing
- Run faster
- Use fewer resources (memory, registers, space, fewer fetches)
- Common optimization
 - ▶ Common sub-expression elimination
 - ▶ Copy propagation
 - ▶ Dead code elimination
 - ▶ Code motion
 - ▶ Strength reduction
 - ▶ Constant folding

$$PI = 3.14159$$

$$Area = 4 * PI * R^2$$

$$Volume = (4/3) * PI * R^3$$

Code Optimization

- Précis writing
- Run faster
- Use fewer resources (memory, registers, space, fewer fetches)
- Common optimization
 - ▶ Common sub-expression elimination
 - ▶ Copy propagation
 - ▶ Dead code elimination
 - ▶ Code motion
 - ▶ Strength reduction
 - ▶ Constant folding

$$PI = 3.14159$$

$$Area = 4 * PI * R^2$$

$$Volume = (4/3) * PI * R^3$$

$$X = R * R$$

$$Area = 12.56636 * X$$

$$Volume = 4.18879 * X * R$$

Code Generation

- Usually, a two-step process

Code Generation

- Usually, a two-step process
 - ▶ Generate **intermediate code** from the semantic representation of the program.

Code Generation

- Usually, a two-step process
 - ▶ Generate **intermediate code** from the semantic representation of the program.
 - ▶ Generate machine code from the intermediate code

Code Generation

- Usually, a two-step process
 - ▶ Generate **intermediate code** from the semantic representation of the program.
 - ▶ Generate machine code from the intermediate code
- Each phase will be simple

Code Generation

- Usually, a two-step process
 - ▶ Generate **intermediate code** from the semantic representation of the program.
 - ▶ Generate machine code from the intermediate code
- Each phase will be simple
- $m * n$ vs $m + n$ problem

Design of Intermediate Language

Design of Intermediate Language

Level of abstraction

Source Level	Target Level
Identifier	Memory
Expression	Register
Operator	Stack
Statement	Opcode
Condition	Addressing Mode
Functions	Libraries
Iteration	OS interaction

Design of Intermediate Language

Level of abstraction

Source Level	Target Level
Identifier	Memory
Expression	Register
Operator	Stack
Statement	Opcode
Condition	Addressing Mode
Functions	Libraries
Iteration	OS interaction

- IR semantics should ideally be independent of both the source and target language

Design of Intermediate Language

Level of abstraction

Source Level	Target Level
Identifier	Memory
Expression	Register
Operator	Stack
Statement	Opcode
Condition	Addressing Mode
Functions	Libraries
Iteration	OS interaction

- IR semantics should ideally be independent of both the source and target language
- It is next to impossible to design a single intermediate language to accommodate all programming languages.

Design of Intermediate Language

Level of abstraction

Source Level	Target Level
Identifier	Memory
Expression	Register
Operator	Stack
Statement	Opcode
Condition	Addressing Mode
Functions	Libraries
Iteration	OS interaction

- IR semantics should ideally be independent of both the source and target language
- It is next to impossible to design a single intermediate language to accommodate all programming languages.
- However, common IRs for similar languages and similar machines have been designed and are used for compiler development

Code Generation

- Code generation is a mapping from the source-level abstractions to target machine abstractions.

Code Generation

- Code generation is a mapping from the source-level abstractions to target machine abstractions.
- Map identifiers to memory.

Code Generation

- Code generation is a mapping from the source-level abstractions to target machine abstractions.
- Map identifiers to memory.
- Map operators to opcode(s).

Code Generation

- Code generation is a mapping from the source-level abstractions to target machine abstractions.
- Map identifiers to memory.
- Map operators to opcode(s).
- Convert conditionals and iterations to a test/jump or compare instructions

Code Generation

- Code generation is a mapping from the source-level abstractions to target machine abstractions.
- Map identifiers to memory.
- Map operators to opcode(s).
- Convert conditionals and iterations to a test/jump or compare instructions
- Layout parameter passing protocols: locations for parameters, return values, etc.

Code Generation

- Code generation is a mapping from the source-level abstractions to target machine abstractions.
- Map identifiers to memory.
- Map operators to opcode(s).
- Convert conditionals and iterations to a test/jump or compare instructions
- Layout parameter passing protocols: locations for parameters, return values, etc.
- Interface calls to library, runtime systems, and operating systems

Symbol Table

Symbol Table

- Information required about the program variables during compilation

Symbol Table

- Information required about the program variables during compilation
 - ▶ Class of variable: keyword, identifier, etc.

Symbol Table

- Information required about the program variables during compilation
 - ▶ Class of variable: keyword, identifier, etc.
 - ▶ Type of variable: integer, float, array, function, etc.

Symbol Table

- Information required about the program variables during compilation
 - ▶ Class of variable: keyword, identifier, etc.
 - ▶ Type of variable: integer, float, array, function, etc.
 - ▶ Amount of storage required

Symbol Table

- Information required about the program variables during compilation
 - ▶ Class of variable: keyword, identifier, etc.
 - ▶ Type of variable: integer, float, array, function, etc.
 - ▶ Amount of storage required
 - ▶ Address in the memory

Symbol Table

- Information required about the program variables during compilation
 - ▶ Class of variable: keyword, identifier, etc.
 - ▶ Type of variable: integer, float, array, function, etc.
 - ▶ Amount of storage required
 - ▶ Address in the memory
 - ▶ Scope information

Symbol Table

- Information required about the program variables during compilation
 - ▶ Class of variable: keyword, identifier, etc.
 - ▶ Type of variable: integer, float, array, function, etc.
 - ▶ Amount of storage required
 - ▶ Address in the memory
 - ▶ Scope information
- Two ways to store this data

Symbol Table

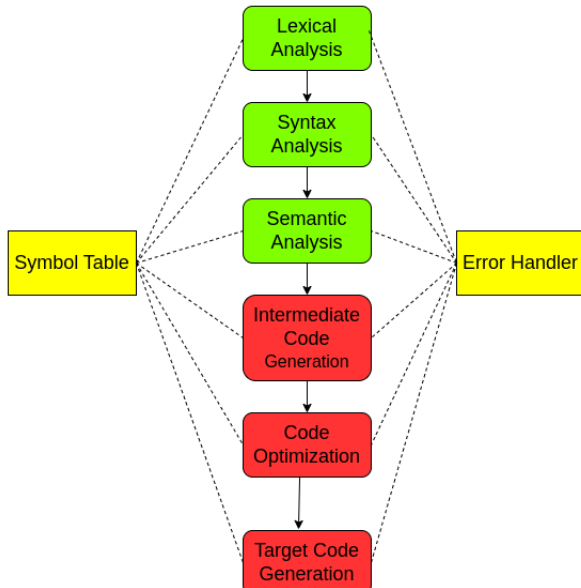
- Information required about the program variables during compilation
 - ▶ Class of variable: keyword, identifier, etc.
 - ▶ Type of variable: integer, float, array, function, etc.
 - ▶ Amount of storage required
 - ▶ Address in the memory
 - ▶ Scope information
- Two ways to store this data
 - ▶ Attach the attribute with the symbols

Symbol Table

- Information required about the program variables during compilation
 - ▶ Class of variable: keyword, identifier, etc.
 - ▶ Type of variable: integer, float, array, function, etc.
 - ▶ Amount of storage required
 - ▶ Address in the memory
 - ▶ Scope information
- Two ways to store this data
 - ▶ Attach the attribute with the symbols
 - ▶ Store it at a separate location.

Final Compiler Structure

Final Compiler Structure



Advantage of this model(Analysis-Synthesis model)

Advantage of this model(Analysis-Synthesis model)

- Front-end phases are known as analysis phases

Advantage of this model(Analysis-Synthesis model)

- Front-end phases are known as analysis phases
- Back-end phases are known as synthesis phases

Advantage of this model(Analysis-Synthesis model)

- Front-end phases are known as analysis phases
- Back-end phases are known as synthesis phases
- Each phase has a well-defined work

Advantage of this model(Analysis-Synthesis model)

- Front-end phases are known as analysis phases
- Back-end phases are known as synthesis phases
- Each phase has a well-defined work
- Each phase handles a logical activity in the process of compilation

Advantage of this model(Analysis-Synthesis model)

- Front-end phases are known as analysis phases
- Back-end phases are known as synthesis phases
- Each phase has a well-defined work
- Each phase handles a logical activity in the process of compilation
- Source and machine-independent code optimization is possible.

Advantage of this model(Analysis-Synthesis model)

- Front-end phases are known as analysis phases
- Back-end phases are known as synthesis phases
- Each phase has a well-defined work
- Each phase handles a logical activity in the process of compilation
- Source and machine-independent code optimization is possible.
- The optimization phase can be inserted after the front and back end phases have been developed and deployed

How can we trust compiler-generated code?

How can we trust compiler-generated code?

- Prove that the compiler is correct.

How can we trust compiler-generated code?

- Prove that the compiler is correct.
- No such technique proves such a large piece of code.

How can we trust compiler-generated code?

- Prove that the compiler is correct.
- No such technique proves such a large piece of code.
- Do systematic testing to increase the confidence level

How can we trust compiler-generated code?

- Prove that the compiler is correct.
- No such technique proves such a large piece of code.
- Do systematic testing to increase the confidence level
- Regression testing

How can we trust compiler-generated code?

- Prove that the compiler is correct.
- No such technique proves such a large piece of code.
- Do systematic testing to increase the confidence level
- Regression testing
 - ▶ Maintain a suite of test programs

How can we trust compiler-generated code?

- Prove that the compiler is correct.
- No such technique proves such a large piece of code.
- Do systematic testing to increase the confidence level
- Regression testing
 - ▶ Maintain a suite of test programs
 - ▶ Expected behavior of each program is documented

How can we trust compiler-generated code?

- Prove that the compiler is correct.
- No such technique proves such a large piece of code.
- Do systematic testing to increase the confidence level
- Regression testing
 - ▶ Maintain a suite of test programs
 - ▶ Expected behavior of each program is documented
 - ▶ All the test programs are compiled using the compiler, and deviations are reported to the compiler writer

How can we trust compiler-generated code?

- Prove that the compiler is correct.
- No such technique proves such a large piece of code.
- Do systematic testing to increase the confidence level
- Regression testing
 - ▶ Maintain a suite of test programs
 - ▶ Expected behavior of each program is documented
 - ▶ All the test programs are compiled using the compiler, and deviations are reported to the compiler writer
- Design of test suite

How can we trust compiler-generated code?

- Prove that the compiler is correct.
- No such technique proves such a large piece of code.
- Do systematic testing to increase the confidence level
- Regression testing
 - ▶ Maintain a suite of test programs
 - ▶ Expected behavior of each program is documented
 - ▶ All the test programs are compiled using the compiler, and deviations are reported to the compiler writer
- Design of test suite
 - ▶ Test programs should exercise every statement of the compiler at least once

How can we trust compiler-generated code?

- Prove that the compiler is correct.
- No such technique proves such a large piece of code.
- Do systematic testing to increase the confidence level
- Regression testing
 - ▶ Maintain a suite of test programs
 - ▶ Expected behavior of each program is documented
 - ▶ All the test programs are compiled using the compiler, and deviations are reported to the compiler writer
- Design of test suite
 - ▶ Test programs should exercise every statement of the compiler at least once
 - ▶ Usually requires great ingenuity to design such a test suite